Monash University FIT2102
Assignment 1

Max Jaszewski (31490816)
Sunday 3rd September 2023

Introduction

In the following sections, the design and logic choices for my assignment 1 submission of the Tetris game will be explained and justified or constructively challenged. An emphasis will be placed on how the code successfully implements the functional reactive style of programming (FRP).

Architecture

The high-level architecture of this game is based on the Model-View-Controller (MVC) application pattern. The files in the source code can be logically divided into each of these responsibilities.

File	MVC Component	Description
./main.ts	Controller	Defines mapping of keyboard and time inputs to actions
./state.ts	Model	Defines how inputs modify state
./view.ts	View	Render Model to the View

The MVC architecture is conducive to FRP since its segregation of responsibility allows the components to be divided into functions.

State

To avoid functions that modify existing state in place through side effects, the state is accumulated through an Observable Scan Pipe Operator that applies state modifications to a new version each repeat of the MVC cycle.

A key decision in the design of the state was to represent the Tetris and Static blocks as matrices. The advantages can be summarised as:

- Matrices can be represented as two dimensional JavaScript (JS) arrays which provide built-in filter, map, and reduce functions.
- Key game functionality can be implemented with mathematical matrix functions which are granular and pure.

- Matrices are scalable and matrix functions are reusable, meaning the game can be easily expanded to accommodate different blocks and board sizes.
- Matrix definitions can be written with generic types, meaning functions and data structures can take advantage of parametric polymorphism and be reusable on matrices of any type.

Exceptions to FRP

It is important to declare that impure functions have been written in the view.ts file, where it is necessary to edit the external properties of the HTML DOM; however, design decisions have been made to reduce side effects.

- Firstly, individual blocks are not identified by an id. Every cycle of the MVC, the board is cleared and then re-rendered from the matrix representations. Although blocks may now be repeatedly removed and repainted when not edited during a cycle, this declarative approach avoids imperative management of individual HTML elements.

- Secondly, the “game over” logical state is managed with a Boolean flag in the state object, where a true value prevents further modifications to the state until reset. The other option was to use an unsubscribe method which had the side effect of unsubscribing from the observable defined in the main.ts.

References

Dwyer, T. (2023, September 3). Asteroids2023
https://stackblitz.com/edit/asteroids2023

Dwyer, T. (2023, August 28). Tim's Code Stuff. FRP Asteroids.
https://tgdwyer.github.io/asteroids/

Dwyer, T. (2023, August 28). Tim's Code Stuff. Functional Reactive Programming
https://tgdwyer.github.io/asteroids/